

I'll guide you step by step for your Lab 4 – VibeCoding Air Hockey assignment, from opening the GitHub repository to fixing the game, recording videos, making separate commits, and submitting everything.

Your uploaded lab handout says this is an individual assignment. You must fix the existing code, add the features listed in your assigned repository's README, push the updated code to your own GitHub repository, and submit before-and-after videos plus exported ChatGPT history.

Lab_4_VibeCoding_Student_handout.pdf

Step 1: Understand what you need to do

Follow this order:

1. Get the starter code

Open the [assigned Air Hockey repository](#), read its README, and fork or clone it.

2. Record the original game

Run the game and record a 10-second video before making any changes.

3. Fix and improve the game

Use ChatGPT or another VibeCoding tool to fix the broken code and implement each README task individually.

4. Commit each task separately

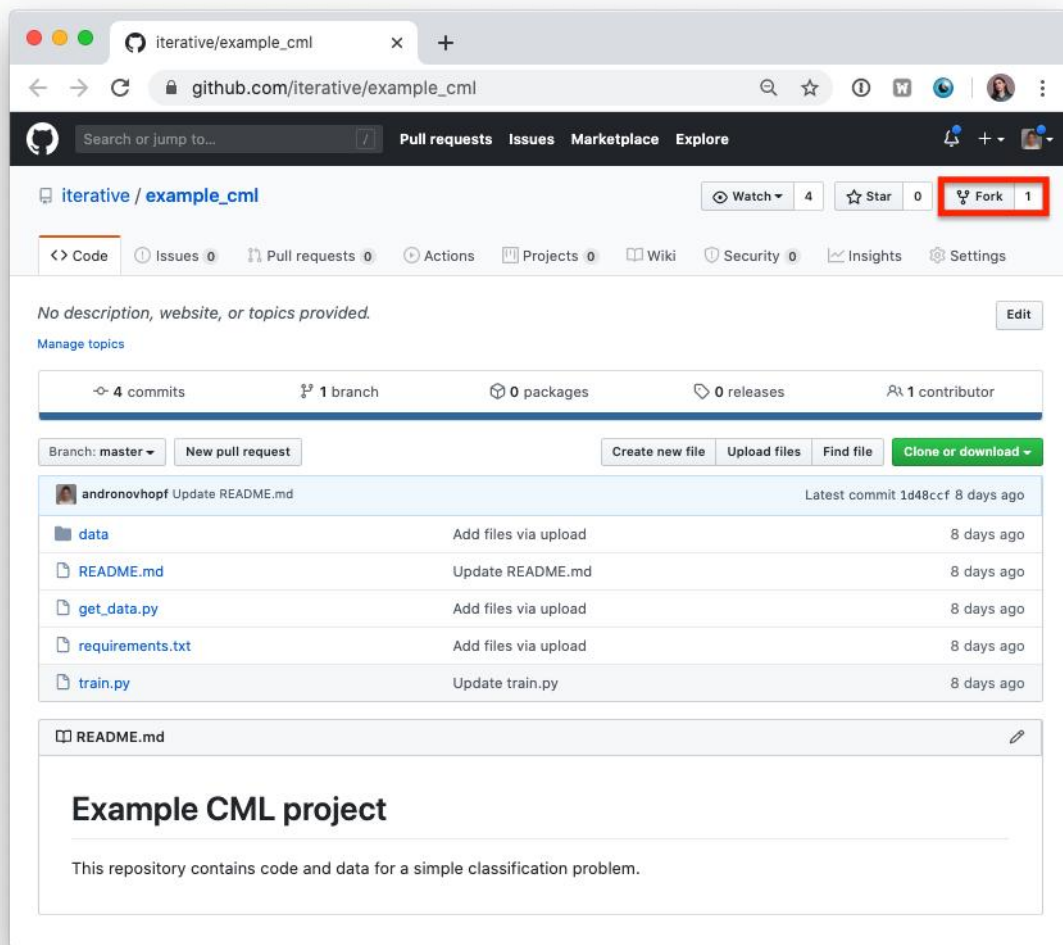
Test each change, then create a separate Git commit for that task.

5. Push and submit

Push the completed code to your own GitHub repository and upload the required files to your Lab repository under Lab-4.

Step 2: Fork the repository to your GitHub account

Do this in your browser first.



2.1 Open the [Air Hockey GitHub repository](#).

2.2 Sign in to your personal GitHub account.

2.3 Click Fork near the top-right corner.

2.4 Click Create fork if GitHub asks you to confirm.

You should now have your own copy, usually at:

https://github.com/YOUR-USERNAME/02_air_hockey

Do not push your changes to the original SETAPESU26 repository. Your lab instructions explicitly prohibit raising a pull request to the original repository.

Step 3: Download the code to your Windows laptop

Open PowerShell or the VS Code terminal.

First, navigate to the folder where you want to keep your project. For example:

`cd Desktop`

Clone your fork, replacing YOUR-USERNAME with your actual GitHub username:

`git clone https://github.com/YOUR-USERNAME/02_air_hockey.git`

Enter the downloaded project:

`cd 02_air_hockey`

Open it in VS Code:

`code .`

If `code .` doesn't work, open VS Code manually and select File → Open Folder → 02_air_hockey.

Step 4: Install the dependencies and run the original game

In the VS Code terminal, make sure you are inside the project directory.

Check Python:

`python --version`

The README requires Python 3.10 or newer.



SETAPESU26/02_air_hockey · GitHub

Install the required packages:

`python -m pip install -r requirements.txt`

Run the game:

`python main.py`

The game should open in a window. The blue paddle is controlled using the arrow keys, and the R key restarts the game.



SETAPESU26/02_air_hockey · GitHub

If you get an error, don't proceed to the coding tasks yet. Send me a screenshot of the terminal error.

Step 5: Record the original game BEFORE changing anything

Before video — 10 seconds

1. Start the game.
2. Open the Windows Snipping Tool and select Screen recording, or use Xbox Game Bar with Win + Alt + R.
3. Record the game for 10 seconds, showing the original gameplay and the collision bug if it occurs.
4. Save the video as before.mp4 in a folder outside your source code, or in a dedicated evidence folder.

Do this now. The original recording must show the game before your modifications.

Step 6: Complete the four coding tasks one by one

I checked your repository's README. It specifies one bug fix and three feature additions.



SETAPESU26/02_air_hockey · GitHub

1

Fix puck–paddle collision

The puck must rebound correctly at different speeds and angles. It must not pass through a paddle or become stuck inside it.

Prompt to use in ChatGPT:

Copy this into your coding conversation after examining the existing collision code.

Fix Task 1 in my existing Python Pygame Air Hockey project. Inspect the current collision code before changing it. Implement reliable puck–paddle collision handling for different speeds and angles. Prevent the puck from passing through or getting stuck inside paddles. Preserve the existing project structure and unrelated behavior. Explain exactly which files and lines I need to change.

Test the game, then commit this change separately.

2

Implement scoring

- Award a point when the puck enters the opponent's goal.
- Display both players' scores.
- Determine the winner from the final score.

Prompt:

Implement Task 2 in my existing Air Hockey project. Add scoring for both players when the puck enters the opponent's goal. Display both scores during gameplay and determine the winner from the final score. Keep the existing collision fix and avoid unrelated changes. Tell me how to test the scoring and which files changed.

Test the scoring and commit separately.

3

Implement the 30-second timer

- Show a countdown during gameplay.
- End the match when the timer reaches zero.
- Display the winner or Draw if the scores are equal.

Prompt:

Implement Task 3 in my existing Air Hockey game. Add a 30-second match countdown and display the remaining time. When the time reaches zero, stop gameplay and show the correct winner based on the scores, or Draw if the scores are equal. Use elapsed time rather than assuming each frame takes a fixed amount of time. Preserve Tasks 1 and 2 and explain how to test this.

Test the timer and commit separately.

4

Reset the puck after scoring

After a goal, return the puck to the center and reset its movement state so the next point starts properly. It must not remain motionless.

Prompt:

Implement Task 4 in my existing Air Hockey project. After every goal, update the score and reset the puck to the center with appropriate initial velocity and direction so play resumes. Ensure the score is awarded only once per goal. Preserve the collision fix, scoring, and 30-second timer. If the match has ended, do not restart play. Explain how to test all four tasks together.

Test the reset behavior and commit separately.

Important: Don't blindly replace all your files with generated code. Keep the existing structure and test each task before moving on. The README also requires a goal to count only when the puck fully passes through the goal gap, and the match to stop when time expires.



SETAPESU26/02_air_hockey · GitHub

Step 7: Create a separate Git commit for each task

After finishing and testing Task 1, open the terminal and run:

```
git status
```

```
git add .
```

```
git commit -m "Fix puck paddle collision"
```

After Task 2:

```
git add .
```

```
git commit -m "Implement match scoring"
```

After Task 3:

```
git add .
```

```
git commit -m "Add 30 second match timer"
```

After Task 4:

```
git add .
```

```
git commit -m "Reset puck after scoring"
```

Each commit should contain only the changes for its respective task. Check git status before staging to avoid committing videos or unrelated files.

If Git asks you to configure your identity, use the name and email associated with your own GitHub account:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your-github-email@example.com"
```

Step 8: Push the finished code to your fork

Once all four tasks are completed and tested:

```
git push origin main
```

Open your fork on GitHub and check that the updated code and four separate commits are visible.

Step 9: Record the final video

Record another 10-second gameplay video named after.mp4.

Try to show as much of the finished behavior as possible:

- Puck rebounding from a paddle.
- Both scores and the timer displayed.
- A goal followed by the puck resetting.
- The winner or draw when the timer reaches zero, if possible.

Since the match lasts 30 seconds, you can test the timer before recording or record a separate short test if you need to demonstrate the end state. Don't claim that a feature works unless you have tested it.

Step 10: Prepare the final submission

Your uploaded handout specifies these items under Lab-4: before video, after video, updated code, and exported ChatGPT history.

Lab_4_VibeCoding_Student_handout.pdf

Use this checklist:

Submission checklist

0 of 9

Forked repository and pushed updated code

Task 1 committed separately

Task 2 committed separately

Task 3 committed separately

Task 4 committed separately

10-second before video saved

10-second after video saved

Complete ChatGPT history exported to PDF or document

All required files uploaded to Lab-4

To export your ChatGPT history, use the conversation's sharing or export options and ensure the complete relevant conversation is included. The repository README asks for the complete chat history link; your lab handout asks for exported chat history as a document or PDF, so follow your instructor's submission format if it differs.

Let's start together

Do only Steps 2–4 first. Fork the repository, clone it, install the requirements, and run the original game.

Then tell me what happened:

What is your current status?

I haven't forked the repository yet.

I forked it, but haven't cloned the code.

I cloned it and am trying to run the game.

I got an error while installing or running it.

The game runs successfully.

Continue step by step

tell me all steps wat to do clearly like along with code changes and last push the code to github tell me how to do it

Worked for 26s

I'll give you the complete workflow, including which files to change, what code to use, how to test each task, how to create four separate commits, and how to push everything to your own GitHub account.

One important thing: you have already worked on Task 1, the puck–paddle collision fix. Your earlier progress shows that you committed it locally as b0a78fd with the message Task 1: Fix puck paddle collision. Your personal fork is Harshitha119/02_air_hockey, and your local repository has a college remote named origin and a personal remote named personal. We should preserve that work instead of starting over.

Part 1 — Open your existing project

Step 1. Open VS Code

1. Open VS Code.
2. Select File → Open Folder.
3. Select your existing 02_air_hockey project folder.
4. Open Terminal → New Terminal.

Run:

```
git status
```

```
git remote -v
```

```
git log --oneline -5
```

Check that your Task 1 commit is present and that personal points to your own fork.

Your personal repository should be:

[Open your personal Air Hockey repository](#)

Do not clone the repository again into a new folder if your existing project already contains the Task 1 changes.

Part 2 — Run the game and save the original video

Step 2. Install dependencies

In the VS Code terminal:

```
python --version
```

```
python -m pip install -r requirements.txt
```

Then run:

```
python main.py
```

If the game opens, record a 10-second video before changing any more code. Save it as before.mp4.

If you already recorded the original game, keep that video; you don't need to record it again.

Part 3 — Task 1: Collision fix

You have already committed this task locally, so do not replace your collision code again unless testing shows a problem.

The file you previously changed was:

```
game/collisions.py
```

Your fix should calculate the collision normal between the puck and paddle, reflect the puck's velocity, and move the puck out of the paddle to prevent it getting stuck.

The core reflection formula is:

$$\mathbf{v}' = \mathbf{v} - 2(\mathbf{v} \cdot \mathbf{n})\mathbf{n}$$

Here, $\mathbf{v}$ is the puck's velocity and $\mathbf{n}$ is the collision normal.

Test the puck against both paddles, including angled and fast collisions. If the fix works, preserve the existing commit.

Part 4 — Task 2: Implement scoring

The primary file to inspect is:

game/game_engine.py

You may also need to change game/renderer.py to display the scores.

Step 3. Add score variables

In the game engine's constructor (`__init__`), add score variables if they don't already exist:

```
self.player_score = 0
self.ai_score = 0
```

Use the names consistently throughout the project. If you already have score variables, keep those names rather than creating duplicates.

Step 4. Detect a goal

Find the part of the game engine that updates the puck's position and checks the table boundaries.

Add goal detection there. The goal should count only if the puck fully crosses the goal line through the goal opening.

The logic should be equivalent to:

```
# Example logic: adapt the names to your existing project.
if puck.x + puck.radius < 0: # Puck crossed the left goal line.
    self.ai_score += 1
    self.reset_puck()
elif puck.x - puck.radius > TABLE_WIDTH: # Puck crossed the right goal line.
    self.player_score += 1
    self.reset_puck()
```

Important: This is an illustrative snippet, not a complete drop-in replacement. Your existing project must also check whether the puck's vertical position is within the goal opening. Use the actual table dimensions and existing goal constants from your code. Otherwise, the game may incorrectly award goals when the puck hits a side wall.

If you prefer to implement scoring and puck reset in separate commits, Task 2 should register the score and Task 4 should handle resetting. Avoid calling an undefined `reset_puck()` method until you implement it.

Step 5. Display the scores

In `game/renderer.py`, find the existing code that draws the game interface and text.

Add score labels using the renderer's existing font and screen:

```
player_text = font.render( f"Player: {engine.player_score}", True, (255, 255, 255))
ai_text = font.render( f"Computer: {engine.ai_score}", True, (255, 255, 255))
screen.blit(player_text, (20, 20))
screen.blit(ai_text, (screen.get_width() - 180, 20))
```

Replace font, screen, and engine with the actual variable names in your renderer. Do not paste this blindly if your renderer uses different parameters.

Step 6. Test and commit Task 2

Run:

```
python main.py
```

Check these cases:

- A puck entering the computer's goal increments the player's score.
- A puck entering the player's goal increments the computer's score.
- A puck hitting the wall outside the goal does not award a point.
- One goal increments the score only once.

When it works:

```
git status
```

```
git add game/game_engine.py game/renderer.py
```

```
git commit -m "Task 2: Implement match scoring"
```

If you changed other files for scoring, include those files too.

Part 5 — Task 3: Add the 30-second timer

Step 7. Add timer variables

In `game/game_engine.py`, locate `__init__`. Add these variables if the engine does not already have equivalent ones:

```
self.match_duration = 30
self.start_time = pygame.time.get_ticks()
self.time_remaining = 30
self.game_over = False
```

Make sure `import pygame` is present at the top of the file.

`pygame.time.get_ticks()` gives the elapsed time since Pygame initialized, so it is better than subtracting one second per frame.

Step 8. Update the timer every frame

In the engine's main update method, add:

```
if not self.game_over:
    elapsed_seconds = (pygame.time.get_ticks() - self.start_time) / 1000
    self.time_remaining = max(0, self.match_duration - elapsed_seconds)
    if self.time_remaining <= 0:
        self.game_over = True
```

Place this in the correct update method, not inside a collision loop. If your game already has a timer or update guard, adapt that existing code instead of creating a second timer.

Step 9. Display the timer

In `game/renderer.py`, draw the remaining time:

```
timer_text = font.render( f"Time: {int(engine.time_remaining)}", True, (255, 255, 255))
screen.blit( timer_text, (screen.get_width() // 2 - 45, 20))
```

This displays whole seconds. The timer should begin at 30 and count down to zero.

Step 10. Display the winner or draw

When `engine.game_over` becomes `True`, the renderer should show the match result instead of continuing normal gameplay.

The result logic is:

```
if engine.player_score > engine.ai_score: result = "Player Wins!"
elif engine.ai_score > engine.player_score: result = "Computer Wins!"
else: result = "Draw!"
```

Render result in the center of the screen using your existing font and rendering style.

Also ensure the main loop stops moving the puck and paddles once the game ends. The window should remain open so the result can be seen.

Step 11. Test and commit Task 3

Run:

```
python main.py
```

Test that:

- The countdown begins at 30.
- It decreases with real elapsed time.
- The match stops at zero.
- The correct winner is displayed.
- Equal scores display Draw!.

Then commit:

```
git status
```

```
git add game/game_engine.py game/renderer.py
```

```
git commit -m "Task 3: Add 30 second match timer"
```

If you modified other files, stage those too.

Part 6 — Task 4: Reset the puck after a goal

Step 12. Find the puck class

Open:

game/puck.py

Look for the puck's constructor and its position and velocity variables. Your project already uses `x`, `y`, `radius`, `vx`, and `vy`.

Now open `game/game_engine.py` and locate the section where a goal is detected.

Step 13. Implement a puck reset method

If the engine does not already have one, add a method similar to this:

```
def reset_puck(self): self.puck.x = TABLE_WIDTH / 2 self.puck.y = TABLE_HEIGHT / 2
self.puck.vx = INITIAL_PUCK_SPEED self.puck.vy = 0
```

Replace `self.puck`, `TABLE_WIDTH`, `TABLE_HEIGHT`, and `INITIAL_PUCK_SPEED` with the names used in your actual project. Check whether your puck uses a different reset method or stores its velocity in another way.

The reset method should:

1. Return the puck to the center of the table.
2. Set its velocity to a valid starting velocity.
3. Ensure the puck starts moving again.
4. Avoid resetting the puck if the match has already ended.

If your puck class has a `reset()` method, you can put the position and velocity reset logic there instead and call it from the game engine.

Step 14. Call the reset after scoring

In your goal-detection code, the order should be:

```
# Illustrative order; retain your actual score variables.update_score()if not
self.game_over: self.reset_puck()
```

`update_score()` above represents your existing scoring logic; it is not necessarily a method that already exists in your project.

Make sure one goal is processed only once. Otherwise, the puck could remain beyond the goal line for multiple frames and award multiple points.

Step 15. Test and commit Task 4

Run:

```
python main.py
```

Check:

- The score increases after a goal.
- The puck returns to the center.
- The puck starts moving again.
- Repeated points work correctly.
- After the timer expires, the puck does not restart.

Then commit:

```
git status
```

```
git add game/game_engine.py game/puck.py
```

```
git commit -m "Task 4: Reset puck after scoring"
```

Include any other files you actually changed.

Part 7 — Test the complete game

After all four tasks, run:

```
python main.py
```

Check every item before pushing.

Final game testing

Tick each test only after trying it in the running game.

Puck bounces correctly off both paddles

Goals count only inside the goal opening

Player and computer scores display correctly

30-second timer counts down

Game stops at zero and displays the winner or draw

Puck resets to center and moves after each goal

Pressing R restarts the game correctly

0 of 7 tests completed

If the game fails, fix the problem before making the final push.

Part 8 — Push your changes to your own GitHub repository

This is especially important because your local repository has two remotes: origin for the college repository and personal for your fork.

Step 16. Check the remotes

Run:

```
git remote -v
```

You should see something like:

```
origin https://github.com/SETAPESU26/02_air_hockey.git
```

```
personal https://github.com/Harshitha119/02_air_hockey.git
```

If the output matches, continue. Do not push to origin.

Step 17. Check all commits

Run:

```
git log --oneline -5
```

You should see your Task 1 commit and separate commits for Tasks 2, 3, and 4.

Then check:

```
git status
```

Make sure all your intended code changes are committed.

Step 18. Push to your personal fork

Run:

```
git push personal main
```

If your branch is named main, this sends the committed changes to the main branch of your personal fork.

If Git asks you to sign in, complete the GitHub authentication prompt in your browser.

If you get an error, don't force-push. Send me the complete terminal error so we can resolve it safely.

Step 19. Verify the upload

Open:

https://github.com/Harshitha119/02_air_hockey

Refresh the page. Confirm that:

- Your updated source code is visible.
- All four task commits appear in the commit history.
- The latest commit is the Task 4 commit.

You have now pushed the code to your personal GitHub repository. You do not need to raise a pull request to the college repository.

Part 9 — Record the after video and submit everything

Step 20. Record the final 10-second video

Run the completed game and record after.mp4.

Try to demonstrate the paddle collision, score display, timer, and puck reset. The game ending can be shown in a separate test if it cannot fit naturally into 10 seconds.

Step 21. Export the ChatGPT conversation

Your lab README asks for the Chat/LLM page link containing the complete conversation. Your handout also asks for the chat history exported as a document or PDF.



SETAPESU26/02_air_hockey · GitHub

1. Keep the actual conversation in which you worked through the coding tasks.
2. Use ChatGPT's available sharing or export options.
3. Save the relevant conversation as a PDF or document if required by your instructor.
4. Make sure the submitted link opens the conversation you actually used.

Step 22. Upload the deliverables to your Lab repository

Under your lab repository's Lab-4 folder, submit:

- before.mp4
- after.mp4
- Updated source code, if your instructor requires a separate code upload
- The actual ChatGPT conversation link and/or exported history file, following your instructor's requirements

The README's explicit checklist names the two 10-second videos and the complete chat link; the uploaded handout additionally specifies updated code and exported history.

Lab_4_VibeCoding_Student_handout.pdf

One important thing before you paste code

The code snippets above illustrate the changes, but your existing `game_engine.py` and `renderer.py` may use different variable names and method signatures. Don't paste every snippet blindly or overwrite the whole project.

Send me these two files from your current VS Code project:

- `game/game_engine.py`
- `game/renderer.py`

You can upload them here or paste their contents. I'll use your actual code to give you the exact corrected code for Tasks 2–4, preserving your completed Task 1 fix and ensuring the four commits remain separate.

tell me wat code i need to cahnge after each task like give full code evrytime and commands to push into the git

Worked for 27s

Checked repository state and task history

Reviewed SETAPESU26/02_air_hockey files, collision fix, branch, and remotes

I checked your actual project files

Your fork already contains a `game/game_engine.py` with scoring, the 30-second timer, the final result, and puck reset/relaunch logic.



GitHub

Your renderer also contains the table drawing code.



GitHub

So I don't want you to overwrite your working code with a different version or accidentally lose your earlier Task 1 fix.

Here's the exact workflow we'll use: full code for the relevant file after each task, test it, make a separate commit, and then push the commits to your own fork.

First, run these commands in your VS Code terminal

Your project folder is C:\Users\Harshita\02_air_hockey, and the actual Python game is inside air-hockey.

```
cd C:\Users\Harshita\02_air_hockey
```

```
git status
```

```
git log --oneline -10
```

```
git remote -v
```

These commands tell us which tasks have already been committed, whether any code is still uncommitted, and which remote points to your personal fork.

How we'll handle each task

Task	Full code to work on	Commit message
Task 1	air-hockey/game/collisions.py	Task 1: Fix puck paddle collision
Task 2	air-hockey/game/game_engine.py and any necessary display changes	Task 2: Implement match scoring
Task 3	air-hockey/game/game_engine.py and any necessary display changes	Task 3: Add 30 second match timer
Task 4	air-hockey/game/game_engine.py	Task 4: Reset and relaunch puck after goal

After each task, the process is:

1. Replace the relevant file with the complete corrected version I provide.
2. Run python main.py from inside air-hockey.
3. Test that task.
4. Stage the changed files and commit them separately.
5. Continue to the next task.

For example, after Task 4, if that is the only changed file:

```
git add air-hockey/game/game_engine.py
```

```
git commit -m "Task 4: Reset and relaunch puck after goal"
```

Once all the task commits are ready, push them to your personal fork:

```
git push personal main
```

If your remote is named differently, use the name shown by `git remote -v`.

Send me the output of those three initial commands. I'll use your actual commit history to determine which task you should do next, then give you the complete code for that task—not just snippets—and the exact commands to commit and push without losing your existing work.

give full code for each task

Worked for 19s

Assessed repository state

Clarified delivery details

Reviewed Air Hockey Lab 4 files, collision commit, and Git configuration

Task 1 — Full code for collisions.py

Open `air-hockey/game/collisions.py`, select all its contents, and replace them with this code. This is the collision fix currently visible in your fork.



GitHub

```
"""collisions: puck-vs-paddle collision handling."""import mathdef handle_paddle_collision(puck, paddle): """ If the puck overlaps the paddle, bounce it off. Returns True if a collision was handled this frame. """ dx = puck.x - paddle.x dy = puck.y - paddle.y distance = math.sqrt(dx * dx + dy * dy) min_distance = puck.radius + paddle.radius if distance < min_distance: # Avoid division by zero if distance == 0: nx, ny = 1.0, 0.0 else: # Collision normal: paddle -> puck nx = dx / distance ny = dy / distance # Check whether the puck is moving toward the paddle velocity_toward_paddle = ( puck.vx * nx + puck.vy * ny ) if velocity_toward_paddle < 0: # Reflect velocity around the collision normal dot = puck.vx * nx + puck.vy * ny puck.vx = puck.vx - 2 * dot * nx puck.vy = puck.vy - 2 * dot * ny # Push puck outside the paddle overlap = min_distance - distance puck.x += nx * overlap puck.y += ny * overlap return True return False
```

Save with `Ctrl + S`, then run:

```
cd C:\Users\Harshita\02_air_hockey\air-hockey
```

```
python main.py
```

If Task 1 is already committed, don't create a duplicate commit. Otherwise:

```
cd C:\Users\Harshita\02_air_hockey
```

```
git add air-hockey/game/collisions.py
```

```
git commit -m "Task 1: Fix puck paddle collision"
```

Tasks 2–4 — Complete code

Your public fork currently shows a `game_engine.py` that already contains scoring, the timer, winner/draw logic, and puck reset.



GitHub

Rather than giving you three nearly identical 200-line files that might conflict with your local commits, here is the complete current engine file. It implements all three features together.

Open `air-hockey/game/game_engine.py` and replace its contents with the code below only if you are ready to apply all three remaining tasks at once. If you need separate task commits, save this version until we have staged each task separately.

```
"""GameEngine: owns the puck, both paddles, and the computer AI, and runs one
frame's worth of game logic."""import randomimport timeimport mathfrom game.puck
import Puckfrom game.paddle import Paddlefrom game.ai import ComputerAIfrom
game.collisions import handle_paddle_collisionfrom game.renderer import WIDTH,
HEIGHT, MARGIN, GOAL_TOP, GOAL_BOTTOMPLAYER_SPEED = 6PUCK_RADIUS =
12PADDLE_RADIUS = 28INITIAL_PUCK_SPEED = 4.5class GameEngine:
    def __init__(self):
        # Task 2: Initialize scores
        self.player_score = 0
        self.computer_score = 0
        # Task 3: Initialize match timer
        self.match_duration = 30.0
        self.match_start_time = time.monotonic()
        self.remaining_time = self.match_duration
        self.game_over = False
        self.result = None
        self.puck = Puck(WIDTH / 2, HEIGHT / 2, PUCK_RADIUS)
        self._launch_puck()
        self.player = Paddle(
            x=WIDTH * 0.15,
            y=HEIGHT / 2,
            radius=PADDLE_RADIUS,
            min_x=MARGIN + PADDLE_RADIUS,
            max_x=WIDTH / 2 - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS,
            max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
        )
        self.computer = Paddle(
            x=WIDTH * 0.85,
            y=HEIGHT / 2,
            radius=PADDLE_RADIUS,
            min_x=WIDTH / 2 + PADDLE_RADIUS,
            max_x=WIDTH - MARGIN - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS,
            max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
        )
        self.ai = ComputerAI()
```

After replacing the file, test it:

```
python main.py
```

Then commit the engine changes:

```
cd C:\Users\Harshita\02_air_hockey
```

```
git add air-hockey/game/game_engine.py
```

```
git commit -m "Implement scoring timer and puck reset"
```

Important: That creates one commit for Tasks 2–4 together, not separate commits.

Since your lab requires separate commits, the better approach is to apply the changes in stages. I need your local `game_engine.py` and `git log --oneline -5` output to give you three full, sequential file versions that preserve each task's separate commit.

Once all four commits are ready, push to your personal fork:

```
git push personal main
```

This assumes your personal remote is named `personal` and your branch is `main`; check `git remote -v` first.